Programación III
Presentación de Cátedra
Computación

Alejandro Sánchez
aljsanchez@gmail.com

Departamento de Informática
Universidad Nacional de San Luis

San Luis

Programación III
Presentación de Cátedra – Computación

1. Árbol de búsqueda
2. Computación de la recursión
3. Ejercicio 1
4. Computación de la recursión de cola
5. Ejercicio 2
6. Computación de la recursión a izquierda

Sustitución

- Conjunto finito de pares:
  $\{v_1 \to t_1, \ldots, v_i \to t_i, \ldots, v_n \to t_n\}$
  con las variables v_i distintas entre sí
  y cada termino t_i distinto de v_i
- Se puede aplicar a una expresión
  $\mathtt{c} = \mathtt{es_padre}(\mathtt{Y}, \mathtt{X}), \mathtt{es_hombre}(\mathtt{X})$

  $\mathtt{s} = \{\mathtt{Y} \to \mathtt{haran}, \mathtt{X} \to \mathtt{lot}\}$
  $\mathtt{c\ s} = \mathtt{es_padre}(\mathtt{haran}, \mathtt{lot}), \mathtt{es_hombre}(\mathtt{lot})$
- La composición de sustituciones $\mathtt{s}_1 \bullet \mathtt{s}_2$ es asociativa
- La sustitución vacía ϵ es el neutro de la composición
- $\mathtt{c\ s}_1\ \mathtt{s}_2 = \mathtt{c}\ (\mathtt{s}_1 \bullet \mathtt{s}_2)$

Unificador

Sustitución que hace que dos términos sean idénticos
Dados $c_1 = p(a, Y)$ y $c_2 = p(X, f(Z))$

$s_1 = \{X \to a, Y \to f(a), Z \to a\}$ es unificador

ya que $c_1 s_1 = c_2 s_1 = p(a, f(a))$ y
$s_2 = \{X \to a, Y \to f(f(a)), Z \to f(a)\}$ también es unificador

ya que $c_1 s_2 = c_2 s_2 = p(a, f(f(a)))$

Unificador Más General (UMG)

- Cualquier otro se puede obtener por composición
- g es UMG de dos términos

  $s_1 = g \bullet u_1$

  $s_2 = g \bullet u_2$

  para sustituciones u_1, u_2, s_1, s_1
- Ejemplo

  Dados $p(X)$ y $p(f(Y))$

  $g = \{X \rightarrow f(Y)\}$ es UMG, ya que

  $p(X)\ g = p(f(Y))\ g = p(f(Y))$

  $s_1 = g \bullet \{Y \rightarrow a\}$ es unificador, ya que

  $p(X)\ s_1 = p(f(Y))\ s_1 = p(f(a))$

Unificación

- `S=T` para consultar si S y T unifican (*match*)
- Dos terminos `S` y `T` unifican si
  - Si son constantes, unifican si son el mismo objeto

    ?- `eva=eva`

    true
  - Si `S` es una variable y `T` otra cosa, entonces unifican con `S` instanciada a `T`

    ?- `X=eva, X=Y`

    X=eva, Y=eva
  - Si son estructuras, unifican si
    - tiene el mismo functor principal
    - sus componentes correspondientes unifican

    ?- `amigos(pedro,Y)=amigos(X,vicente)`

    X=pedro, Y=vicente

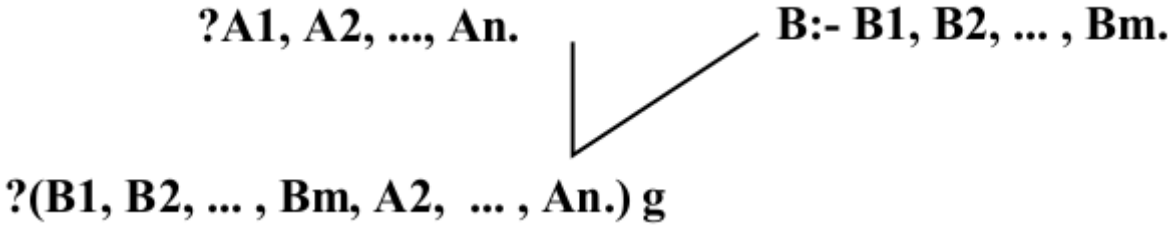
Origen del nombre de la regla de resolución

Resolución SLD

- **S**elective – Se usa regla de selección de claúsula $\mathtt{C_i}$
- **L**inear – La prueba se restringe a una secuencia lineal
  - $\mathtt{C_1}, \mathtt{C_2}, \ldots, \mathtt{C_n}$
  - $\mathtt{C_1}$ es la meta principal
  - $\mathtt{C_2}, \ldots, \mathtt{C_n}$ es el resolvente
  - $\mathtt{C_i}$ es padre de $\mathtt{C_{i+1}}$
  - Se refuta si $\mathtt{C_n}$ es la claúsula vacía
- **D**efinite clause resolution
  - Resolución SL con claúsulas definidas
  - También llamadas claúsulas de Horn estrictas
  - Solo un literal positivo en forma disyuntiva
  - $\neg p_1(\ldots) \vee \neg p_2(\ldots) \vee \ldots \vee \neg p_m(\ldots) \vee p(t_1, t_2, \ldots, t_n)$

Regla de resolución SLD

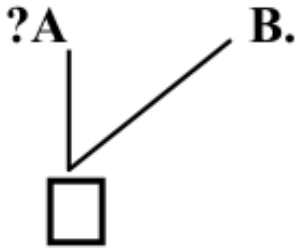
- Dada una consulta $?- A_1, A_2, \ldots, A_n$
- Dada una regla $B :- B_1, B_2, \ldots, B_m.$
- Si $A_1 = B$ (unifican)



- Resolvente: nueva pregunta a resolver

  $?- B_1, B_2, \ldots, B_m, A_2, \ldots, A_n \; g$

- Éxito



Árbol de búsqueda

Dada una consulta $?- \mathtt{A}_1, \mathtt{A}_2, \ldots, \mathtt{A}_n$

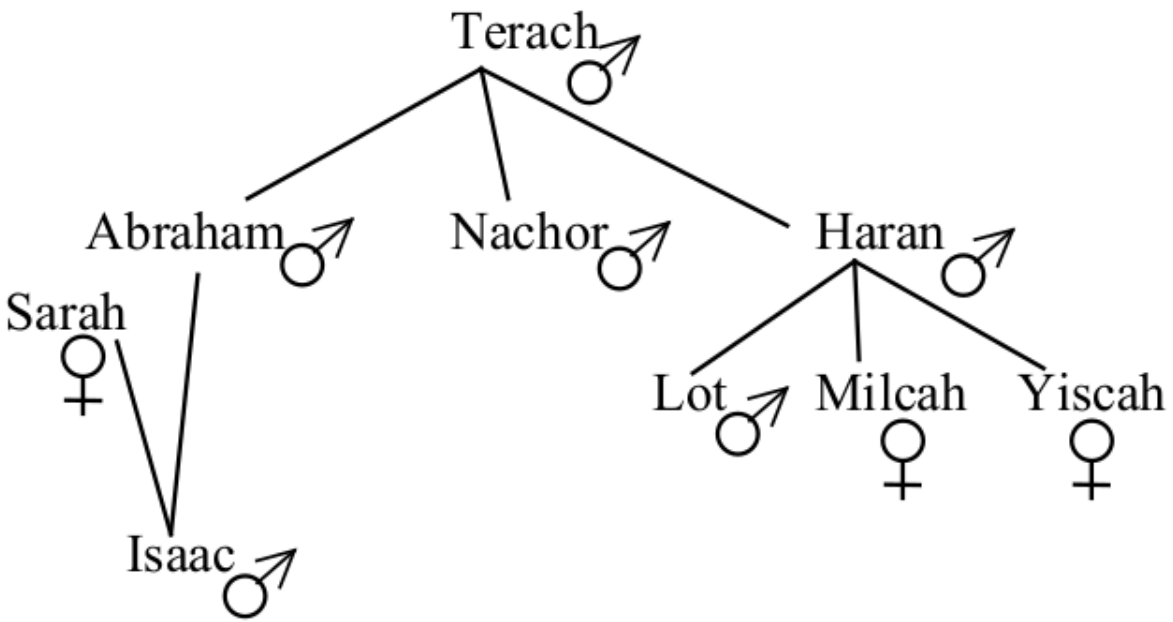
1. Se busca una claúsula con cabeza $\mathtt{B}$ tal que $\mathtt{A}_1 = \mathtt{B}$
   - Se busca desde la primer línea en orden descendiente
   - La consulta fracasa si no se encuentra
2. Se encuentra claúsula con cabeza $\mathtt{B}$ tal que $\mathtt{A}_1 = \mathtt{B}$
   - $\mathtt{A}_1$ se satisface
   - Hecho $\mathtt{B}$
     se busca satisfacer $?- \mathtt{A}_2, \ldots, \mathtt{A}_n$ g
   - Regla $\mathtt{B} :- \mathtt{B}_1, \mathtt{B}_2, \ldots, \mathtt{B}_m.$
     se busca satisfacer $?- \mathtt{B}_1, \mathtt{B}_2, \ldots, \mathtt{B}_m, \mathtt{A}_2, \ldots, \mathtt{A}_n$ g
3. Si $\mathtt{A}_i$ con $i > 1$ fracasa – se hace *backtracking*
   - Se deshace la unificación para $\mathtt{A}_{i-1}$
   - Se busca otra forma de satisfacer $\mathtt{A}_{i-1}$

Descendencia Terach: Hechos

```
1  es_padre(terach, abraham).
2  es_padre(terach, nachor).
3  es_padre(terach, haran).
4  es_padre(abraham, isaac).
5  es_padre(haran, lot).
6  es_padre(haran, milcah).
7  es_padre(haran, yiscah).
8  es_madre(sarah, isaac).
9  es_hombre(terach).
10 es_hombre(abraham).
11 es_hombre(nachor).
12 es_hombre(haran).
13 es_hombre(isaac).
14 es_hombre(lot).
15 es_mujer(sarah).
16 es_mujer(milcah).
17 es_mujer(yiscah).
```

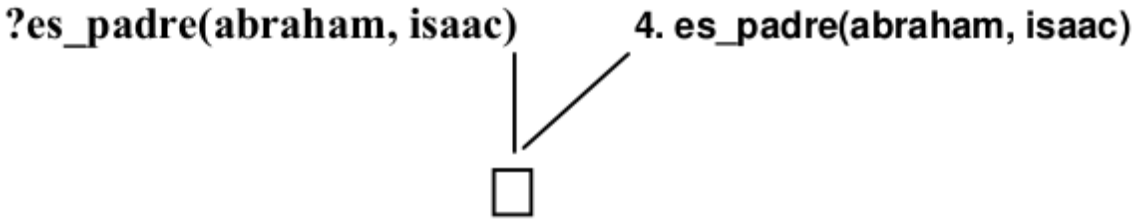
Descendencia Terach: Reglas

```
18 es_hijo(X,Y):- es_padre(Y,X), es_hombre(X).
19 es_hija(X,Y):- es_padre(Y,X), es_mujer(X).
20 es_abuelo(X,Z):- es_padre(X,Y), es_padre(Y,Z).
```



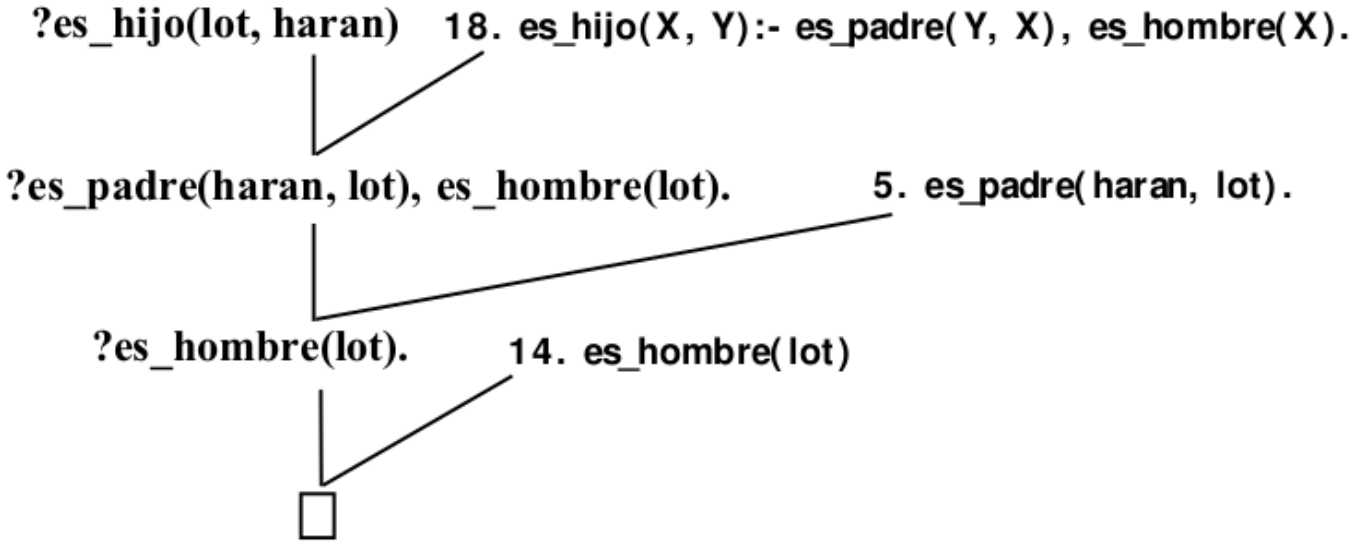
Descendencia Terach: Consulta 1

Resolución



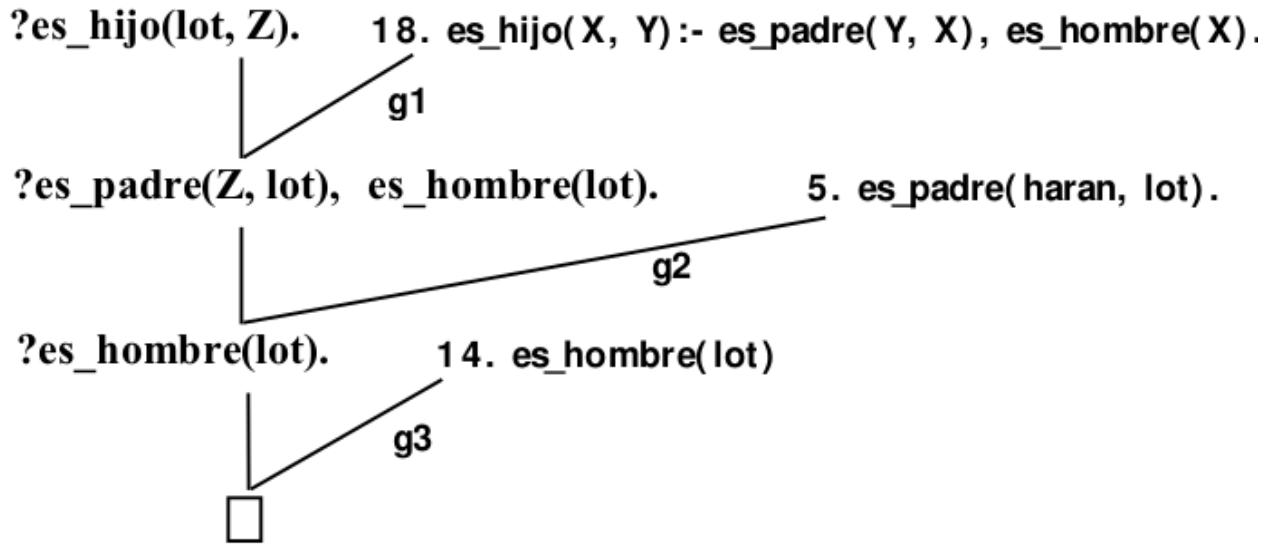
Descendencia Terach: Consulta 2

Resolución



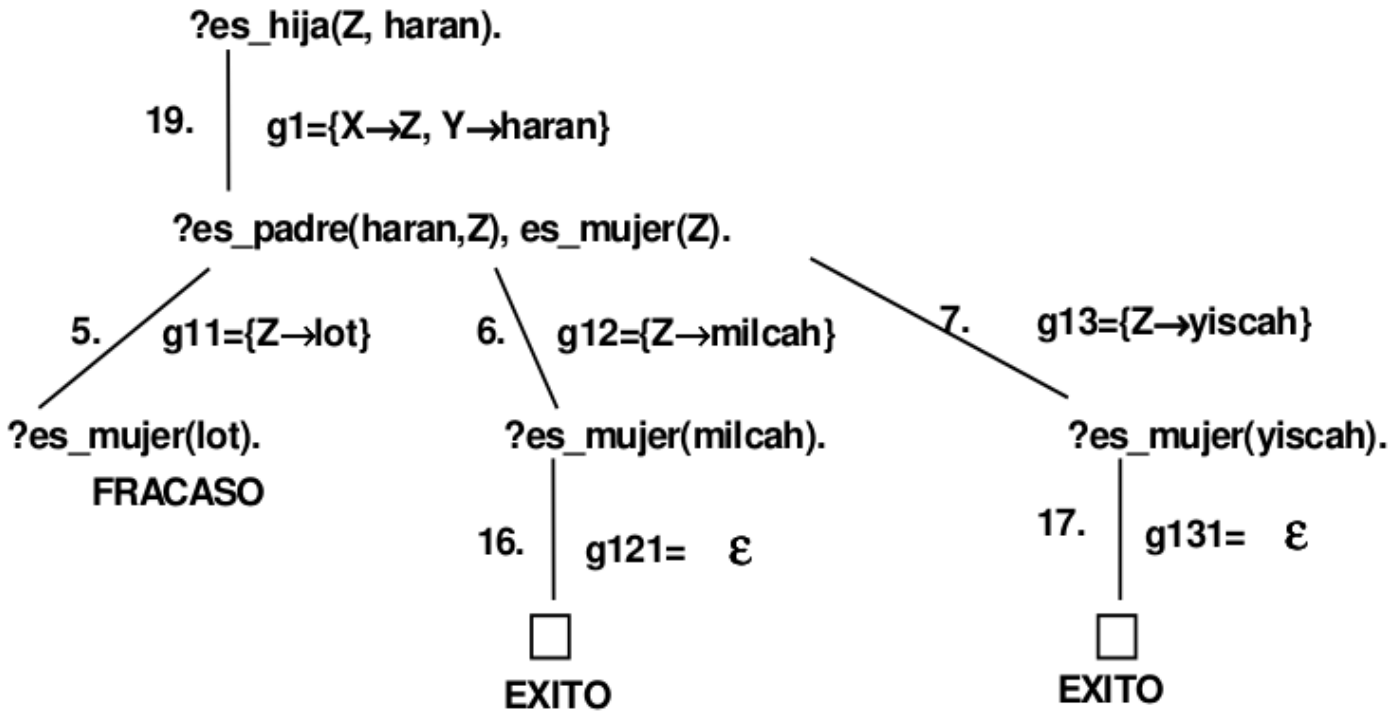
Descendencia Terach: Consulta 3

Resolución



$$g1 = \{X \rightarrow lot, Y \rightarrow Z\}$$
$$g2 = \{Z \rightarrow haran\}$$
$$g3 = \epsilon$$

Descendencia Terach: Consulta 4




```
05. es_padre(haran, lot).
06. es_padre(haran, milcah).
07. es_padre(haran, yiscah).
16. es_mujer(milcah).
17. es_mujer(yiscah).
19. es_hija(X,Y):- es_padre(Y,X), es_mujer(X).
```

Descendencia Terach: Consulta 4

Ejecución

```
→  prolog git:(master) swipl
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.5)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- [terach].
true.

?- es_hija(Z,haran).
Z = milcah ;
Z = yiscah.

?- halt.
→  prolog git:(master)
```

Programación III
Presentación de Cátedra – Computación

1. Árbol de búsqueda
2. Computación de la recursión
3. Ejercicio 1
4. Computación de la recursión de cola
5. Ejercicio 2
6. Computación de la recursión a izquierda

Definiciones recursivas

- Condiciones límites
- Casos recursivos

Miembro de una lista

```
1 miembro(X,[X|Y]).
2 miembro(X,[Y|Z]):-miembro(X,Z).
```

- Condiciones límites
  - Es explícito en la regla 1 cuando se encuentra
  - Es implícito cuando no se encuentra y falla
- Caso recursivo: regla 2
- Modo `miembro`$(?\texttt{Elem}, +\texttt{List})$
- También puede definirse usando una variable muda

```
miembro(X,[X|_]).
miembro(X,[_|Z]):-miembro(X, Z).
```

Miembro de una lista: Consulta 1

```
1 miembro(X,[X|Y]).
2 miembro(X,[Y|Z]):-miembro(X,Z).
```

?- miembro(pedro, [maria, pedro, juan, ana]).

Miembro de una lista: Consulta 1

```
1 miembro(X,[X|Y]).
2 miembro(X,[Y|Z]):-miembro(X,Z).
```

?- miembro(pedro, [maria, pedro, juan, ana]).

? miembro(pedro, [maria, pedro, juan, ana]).

2. | {X1→pedro, Y1→maria, Z1→ [pedro,juan,ana]}

? miembro(pedro, [pedro, juan, ana]).

1. | {X2→pedro, Y2→ [juan,ana]}

□ EXITO (la contestación es **si**)

Miembro de una lista: Consulta 2

```
1 miembro(X,[X|Y]).
2 miembro(X,[Y|Z]):-miembro(X,Z).
```

?- miembro(vicente, [maria, pedro, juan, ana]).

Miembro de una lista: Consulta 2

```
1 miembro(X,[X|Y]).
2 miembro(X,[Y|Z]):-miembro(X,Z).
```

?- miembro(vicente, [maria, pedro, juan, ana]).

? miembro(vicente, [maria, pedro, juan, ana]).

2.

? miembro(vicente, [pedro, juan, ana]).

2.

? miembro(vicente, [juan, ana]).

2.

? miembro(vicente, [ana]).

2.

? miembro(vicente, [ ]).

⊗ FRACASO

Miembro de una lista: Consulta 3

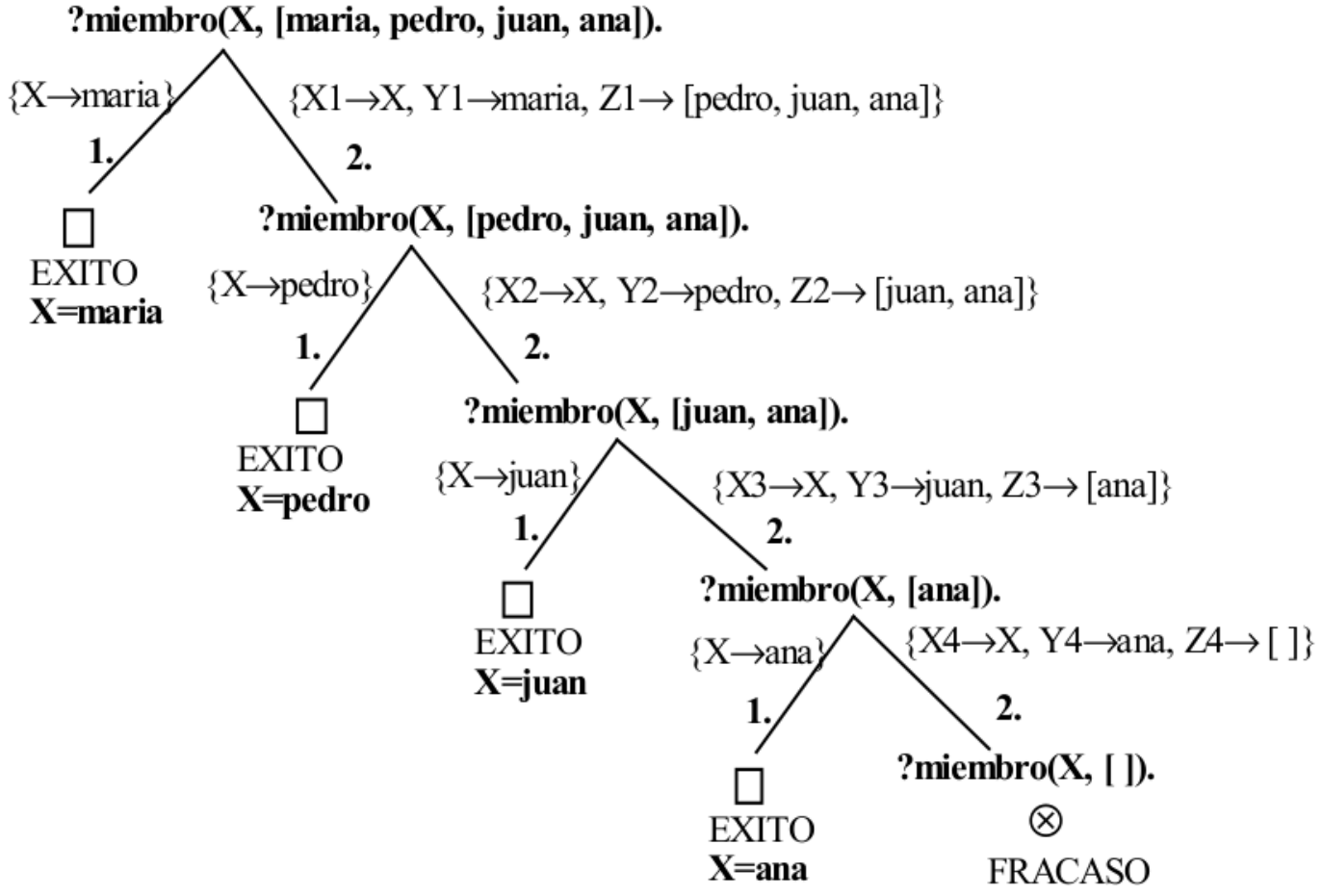
```
1 miembro(X,[X|Y]).
2 miembro(X,[Y|Z]):-miembro(X,Z).
```

?- miembro(X, [maria, pedro, juan, ana]).

Miembro de una lista: Consulta 3

```
1 miembro(X,[X|Y]).
2 miembro(X,[Y|Z]):-miembro(X,Z).
```

?- miembro(X, [maria, pedro, juan, ana]).



Concatenación de listas

```
1 concatenar([],L,L).
2 concatenar([X|Y],Z,[X|U]):-concatenar(Y,Z,U).
```

- Condiciones límites
  - La primera lista es vacía: regla 1
- Caso recursivo: regla 2
- Modo `concatenar`$(?\texttt{List}, ?\texttt{List}, ?\texttt{List})$

Concatenación de listas: Consulta 1

```
1 concatenar([],L,L).
2 concatenar([X|Y],Z,[X|U]):-concatenar(Y,Z,U).
```

?concatenar([a, b, c], [d, e, f], X).

2. **g1**={X1→a, Y1→ [b, c], Z1→ [d, e, f], X→ [a | U1]}

?concatenar([b, c], [d, e, f], U1).

2. **g2** ={X2→b, Y2→ [c], Z2→ [d, e, f], U1→ [b | U2]}

?concatenar([c], [d, e, f], U2).

2. **g3** ={X3→c, Y3→ [ ], Z3→ [d, e, f], U2→ [c | U3]}

?concatenar([ ], [d, e, f], U3).

1. **g4** ={L4→ [d, e, f], U3→ [d, e, f]}

□

EXITO

$\texttt{U3} = [d, e, f]$

$\Longrightarrow \texttt{U2} = [\texttt{c}|\texttt{U3}] = [\texttt{c}, \texttt{d}, \texttt{e}, \texttt{f}]$

$\Longrightarrow \texttt{U1} = [\texttt{b}|\texttt{U2}] = [\texttt{b}, \texttt{c}, \texttt{d}, \texttt{e}, \texttt{f}]$

$\Longrightarrow \texttt{X} = [\texttt{a}|\texttt{U1}] = [\texttt{a}, \texttt{b}, \texttt{c}, \texttt{d}, \texttt{e}, \texttt{f}]$

Programación III
Presentación de Cátedra – Computación

1 Árbol de búsqueda

2 Computación de la recursión

3 Ejercicio 1

4 Computación de la recursión de cola

5 Ejercicio 2

6 Computación de la recursión a izquierda

Concatenación de listas: Ejercicios

```
1 concatenar([],L,L).
2 concatenar([X|Y],Z,[X|U]):-concatenar(Y,Z,U).
```

1. Hacer la resolución SLD de
   1) `?- concatenar([a,b,c],X,[a,b,c,d,e]).`
   2) `?- concatenar([a,b,c],X,[a,c,d,e]).`
2. Hacer el árbol de búsqueda completo de

   `?- concatenar(X,Y,[a,b,c,d,e]).`

Programación III
Presentación de Cátedra – Computación

1 Árbol de búsqueda

2 Computación de la recursión

3 Ejercicio 1

4 Computación de la recursión de cola

5 Ejercicio 2

6 Computación de la recursión a izquierda

Recursión de cola

- La llamada recursiva es la última meta de cualquiera de las reglas del predicado
- Equivale a un comportamiento iterativo
- Elimina necesidad de pila de ejecución
- Algunos ambientes hacen más eficiente la ejecución en tiempo y espacio

Inversa con recursión con acumuladores – n^2

```
1 inversa([],[]).
2 inversa([X|Y],L):-inversa(Y,R),concatenar(R,[X],L).
```

? inversa([a_1, a_2,..., a_n], X).

(1) | **2.** {X1→a_1, Y1→ [a_2, a_3,..., a_n], L1→X}

? inversa([a_2,..., a_n], R1), concatenar(R1, [a_1], X).

(2) | **2.** {X2→a_2, Y2→ [a_3, a_4,..., a_n], L2→R1}

? inversa([a_3,..., a_n], R2), concatenar(R2, [a_2], R1), concatenar(R1, [a_1], X).

•
•
•

(n) | **2.** {Xn→a_n, Yn→ [ ], Ln→Rn-1}

? inversa([ ], R_n), concatenar(R_n, [a_n], R_{n-1}),... , concatenar(R2, [a_2], R1), concatenar(R1, [a_1], X).

(n+1) | **1.** {Rn→ [ ]}

? concatenar([ ], [a_n], R_{n-1}),..., concatenar(R_1, [a_1], X).

(n+2) | **1'.** {Ln+2→ [a_n], Rn-1→ [a_n]}

? concatenar([a_n], [a_{n-1}], R_{n-2}),..., concatenar(R_1, [a_1], X).

(n+3) | **2'.** {Xn+3→ a_n, Yn+3 → [ ], Zn+3 → [a_{n-1}], R_{n-2} → [a_n | Un+3]}

? concatenar([ ], [a_{n-1}], U_{n+3}),..., concatenar(R_1, [a_1], X).

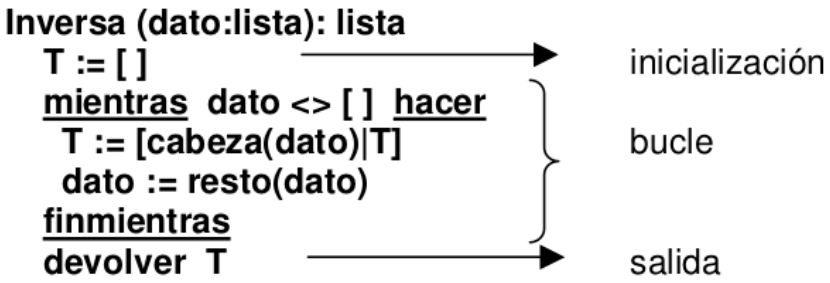
•
•

```
1 concatenar([],L,L).
2 concatenar([X|Y],Z,[X|U]):-concatenar(Y,Z,U).
```

Inversa con recursión de cola – n

```
1 inversa(X,Y):-inversa(X,Y,[]).  % inicialización
2 inversa([X|Z],Y,T):-inversa(Z,Y,[X|T]).  % bucle
3 inversa([],T,T).  % salida
```

Algoritmo simulado



Inversa con recursión de cola – n

```
1 inversa(X,Y):-inversa(X,Y,[]).  % inicialización
2 inversa([X|Z],Y,T):-inversa(Z,Y,[X|T]).  % bucle
3 inversa([],T,T).  % salida
```

? inversa([a_1,...,a_n], X).

(1) **1.** | {X1→ [a_1, ..., a_n], Y1→ X}

? inversa([a_1,...,a_n], X, [ ]).

(2) **2.** | {X2→ a_1, Z2→ [a_2, ..., a_n], Y2→ X, T2→ [ ]}

? inversa([a_2,...,a_n], X, [a_1]).

(3) **2.** | {X3→ a_2, Z3→ [a_3, ..., a_n], Y3→ X, T3→ [a_1]}

? inversa([a_3,...,a_n], X, [a_2, a_1]).

•
•

(n+1) **2.** |

? inversa([ ], X, [a_n,...,a_1]).

(n+2) **3.** | {Tn+2 → [a_n,...,a_1], X → [a_n,...,a_1]}

□

ÉXITO **X=[a_n,...,a_1]**

Programación III
Presentación de Cátedra – Computación

1 Árbol de búsqueda

2 Computación de la recursión

3 Ejercicio 1

4 Computación de la recursión de cola

5 Ejercicio 2

6 Computación de la recursión a izquierda

Ejercicio

- Dada la consulta `?- fact(3,F).`
  1. Hacer árbol de llamadas para cada versión.
  2. Indicar cuál corresponde a recursión de cola

```
1 fact(0,1).
2 fact(N,Y):- M is N-1, fact(M,Z), Y is N*Z.
```

Factorial 1

```
1 fact(X,Y):- factaux(X,1,Y).
2 factaux(0, Y, Y).
3 factaux(N,Y,Z):- U is N*Y, M is N-1, factaux(M,U,Z).
```

Factorial 2

Programación III
Presentación de Cátedra – Computación

1 Árbol de búsqueda

2 Computación de la recursión

3 Ejercicio 1

4 Computación de la recursión de cola

5 Ejercicio 2

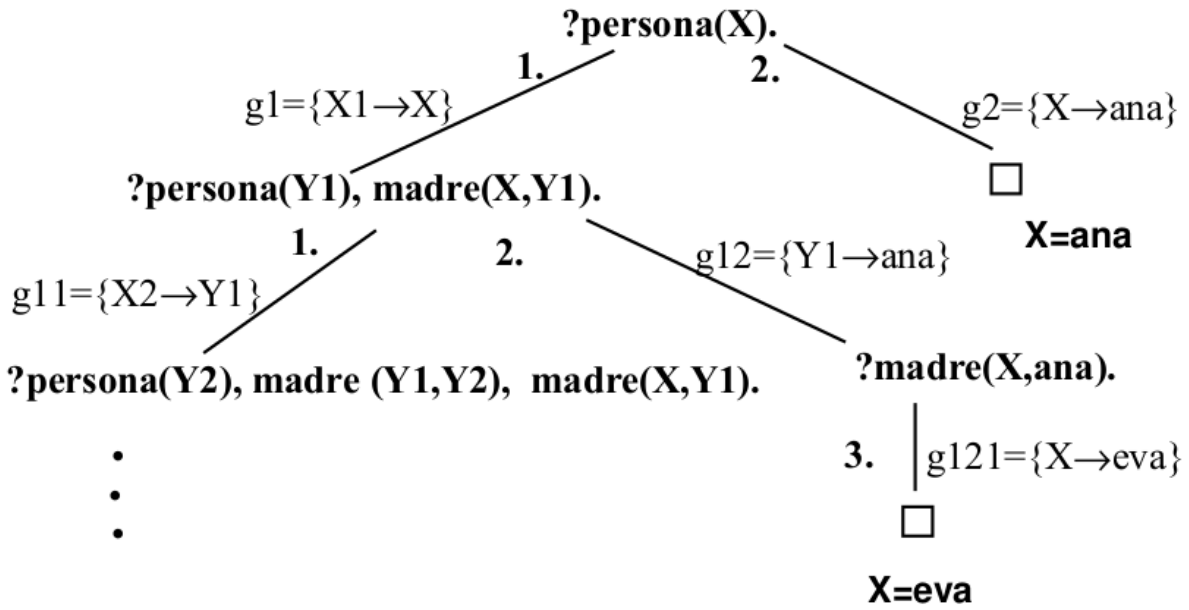
6 Computación de la recursión a izquierda

Recursión a izquierda

- Se produce cuando la llamada recursiva
  - es la primer meta de la regla, y
  - dicha meta es equivalente a la consulta original
- Genera una rama infinita

Recursión infinita a izquierda

```
1 persona(X) :-persona(Y), madre(X,Y).
2 persona(ana).
3 madre(eva, ana).
```



(rama infinita)

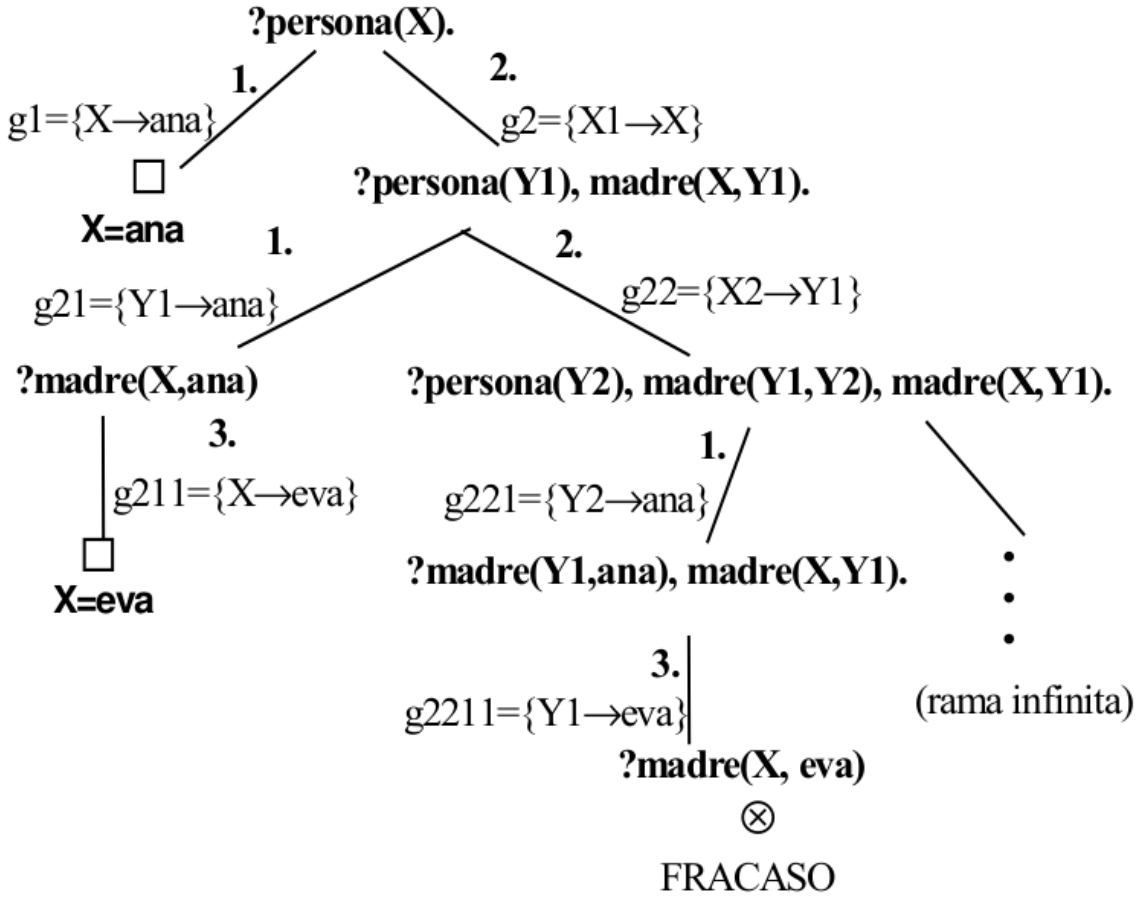
Orden de las reglas

Cambiando el orden de las reglas

- El árbol puede seguir siendo infinito
- Se evita la rama infinita a izquierda
- Permite generar soluciones

Rama izquierda sin recursión infinita

```
1 persona(ana).
2 persona(X):- persona(Y),madre(X,Y).
3 madre(eva,ana).
```

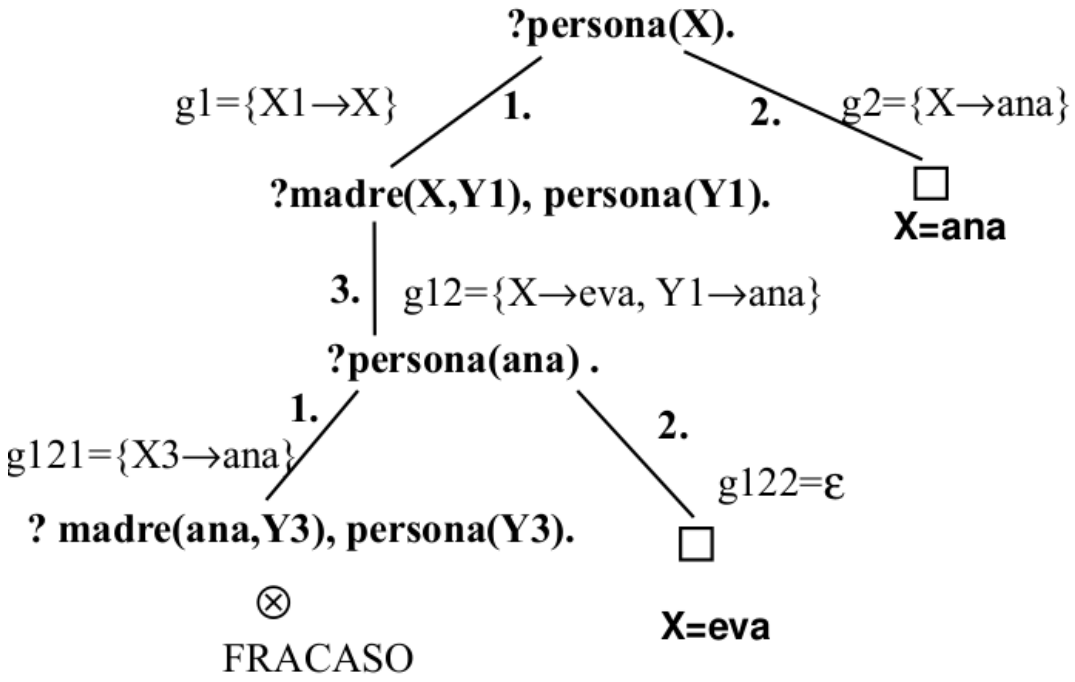


Orden de las metas

- Se cambia el orden de las metas en la regla
- Se evita un árbol infinito

Árbol finito

```
1 persona(X):- madre(X,Y),persona(Y).
2 persona(ana).
3 madre(eva,ana).
```



Generación infinita

- La recursión a izquierda puede generar infinitas soluciones

```
1 generanat(0).
2 generanat(N):- generanat(M), N is M+1.
```

